

Baseball data: Learning Activity 3

Background

This activity uses the **baseball** dataset (available in the *SDAResources* package). The dataset contains statistics on **797 baseball players** from the rosters of all major league teams in November 2004 (Forman, 2004).

You will treat this file as a **population** and draw samples from it using simple random sampling.

Getting Started

Loading the dataset and the R packages.

```
# Uncomment the install.packages("SDAResources") line to install the package.  
# Comment it back after you install the package :)  
# install.packages("SDAResources")  
library(SDAResources)  
library(dplyr)  
library(ggplot2)  
  
data(baseball)
```

Exercise 1: Getting to know the data

Before any analysis, always take a look at the data first.

`head(data)` shows the first 6 rows by default (can change, `head(data, 10)` shows 10).
`dim(data)` tells you the number of rows and columns.

Try them both below. The data name is **baseball** and it is in a format we call a “data frame”.

```
# Include your code here
```

```
head(baseball, 6)
```

```
# A tibble: 6 x 30
```

	team	leagueid	player	salary	pos	gplay	gstart	inning	putout	assist	error
	<chr>	<chr>	<chr>	<dbl>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	ANA	AL	anderga0	6200000	CF	112	92	2375	211	5	2
2	ANA	AL	colonba0	11000000	P	3	34	625	8	30	3
3	ANA	AL	davanje0	375000	CF	108	27	743	75	1	0
4	ANA	AL	donnebr0	375000	P	5	0	126	2	2	0
5	ANA	AL	eckstda0	2150000	SS	142	136	3575	198	309	6
6	ANA	AL	erstada0	7750000	1B	125	124	3196	986	66	4

```
# i 19 more variables: dplay <dbl>, pball <dbl>, gbat <dbl>, atbat <dbl>,  
#   run <dbl>, hit <dbl>, secbase <dbl>, thirdbase <dbl>, homerun <dbl>,  
#   rbi <dbl>, stolenb <dbl>, csteal <dbl>, walk <dbl>, strikeout <dbl>,  
#   intwalk <dbl>, hbp <dbl>, sacrhit <dbl>, sacrfly <dbl>, gidplay <dbl>
```

```
dim(baseball)
```

```
[1] 797 30
```

Exercise 2: Learning how to subset, generate numbers, and take an SRS

Suppose you wanted to select the 1st and 5th row from the baseball data frame and save it as `baseball_subset`. You would:

```
baseball_subset <- baseball[c(1, 5), ]
```

```
# c() creates a vector of values, e.g. c(1, 5) or c(1, 2, 3, 4, 5).  
# baseball is a 2D data frame with rows and columns.  
# baseball[c(1, 5), ] filters the data to keep only the 1st and 5th rows.  
# baseball[, 5] keeps only the 5th column (keeping all the rows).
```

```
baseball_subset
```

```
# A tibble: 2 x 30
```

	team	leagueid	player	salary	pos	gplay	gstart	inning	putout	assist	error
	<chr>	<chr>	<chr>	<dbl>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>

```

1 ANA    AL      anderga0 6200000 CF      112     92   2375     211      5      2
2 ANA    AL      eckstda0 2150000 SS      142    136   3575     198    309     6
# i 19 more variables: dplay <dbl>, pball <dbl>, gbat <dbl>, atbat <dbl>,
#   run <dbl>, hit <dbl>, secbase <dbl>, thirdbase <dbl>, homerun <dbl>,
#   rbi <dbl>, stolenb <dbl>, csteal <dbl>, walk <dbl>, strikeout <dbl>,
#   intwalk <dbl>, hbp <dbl>, sacrhit <dbl>, sacrfly <dbl>, gidplay <dbl>

```

The function `sample(x, size, replace = FALSE)` draws `size` elements from the vector `x`, either with replacement (`replace = TRUE`) or without replacement (`replace = FALSE`). For example, if I wanted to select 5 numbers from 1 to 5 without replacement I would:

```

set.seed(2)
sample(1:5, 5, replace = FALSE)

```

```
[1] 5 3 2 4 1
```

```

# set.seed(number) makes your sample reproducible.
# It will always produce the same result each time you run this code.
# number1:number2 creates a sequence from number1 to number2 with step size 1.

```

Take an SRS of 150 players from the baseball data frame without replacement. Save your sample as `baseball_srs` for use in future exercises. Don't forget to set your seed.

```

# Include your code here

set.seed(2917)
baseball_srs <- baseball[sample(1:nrow(baseball), 150, replace = FALSE),]

```

In words, describe the process of selecting an SRS below:

Exercise 3: Mutating variables and creating histograms

`mutate()` creates new columns that are functions of existing variables. Think of it as adding a new column to your spreadsheet based on existing ones. For example, let's create a new binary column for `run > 50`:

```
baseball <- baseball %>% # this is a fancy piping operator
  mutate(run_above50 = run > 50)

# View the results
baseball %>%
  select(run, run_above50) %>% # selects columns with these names
  head(5)
```

```
# A tibble: 5 x 2
  run run_above50
<dbl> <lgl>
1    57 TRUE
2     0 FALSE
3    41 FALSE
4     0 FALSE
5    92 TRUE
```

```
# You can also select a column like this using the $ symbol

baseball$run[1:5] #selecting the first 5 values of run
```

```
[1] 57  0 41  0 92
```

Salaries are often right-skewed, with a small number of individuals earning substantially more than the rest. Taking the natural logarithm can make the distribution more symmetric, which is helpful when constructing confidence intervals that rely on the Central Limit Theorem. Create a new column in your **baseball_srs** called **logsal** = **log(salary)**. Construct a histogram of the variables **salary** and **logsal** from your SRS using the **hist()** function.

```
# Include your code here. The function hist() will help you plot a basic histogram.

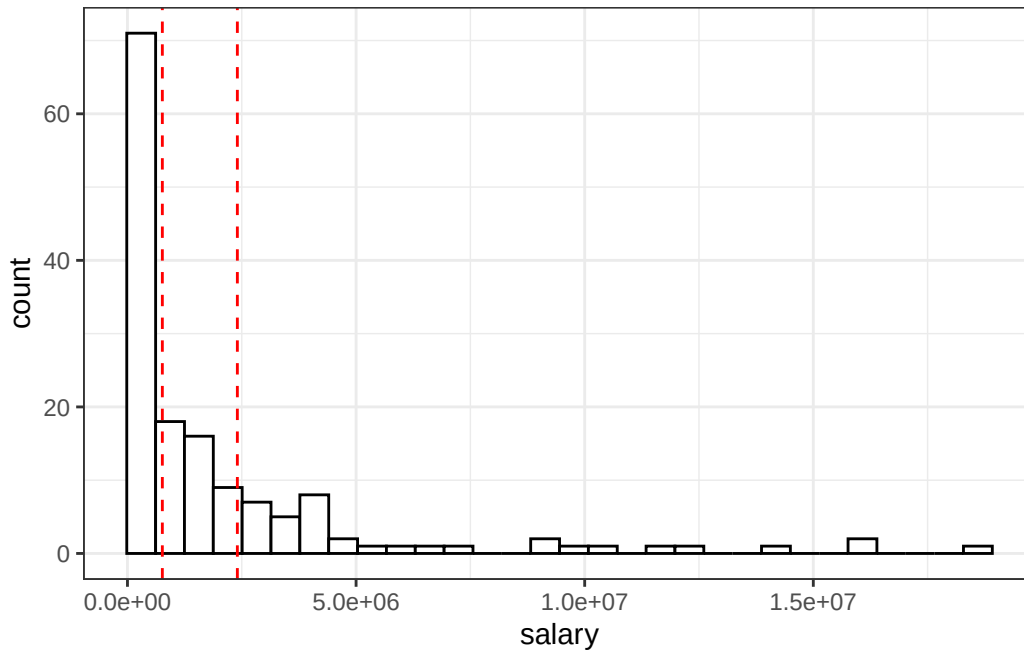
baseball_srs <- baseball_srs %>%
  mutate(logsal = log(salary))

quantile(baseball_srs$salary)
```

0%	25%	50%	75%	100%
300000	320500	762500	2403125	18600000

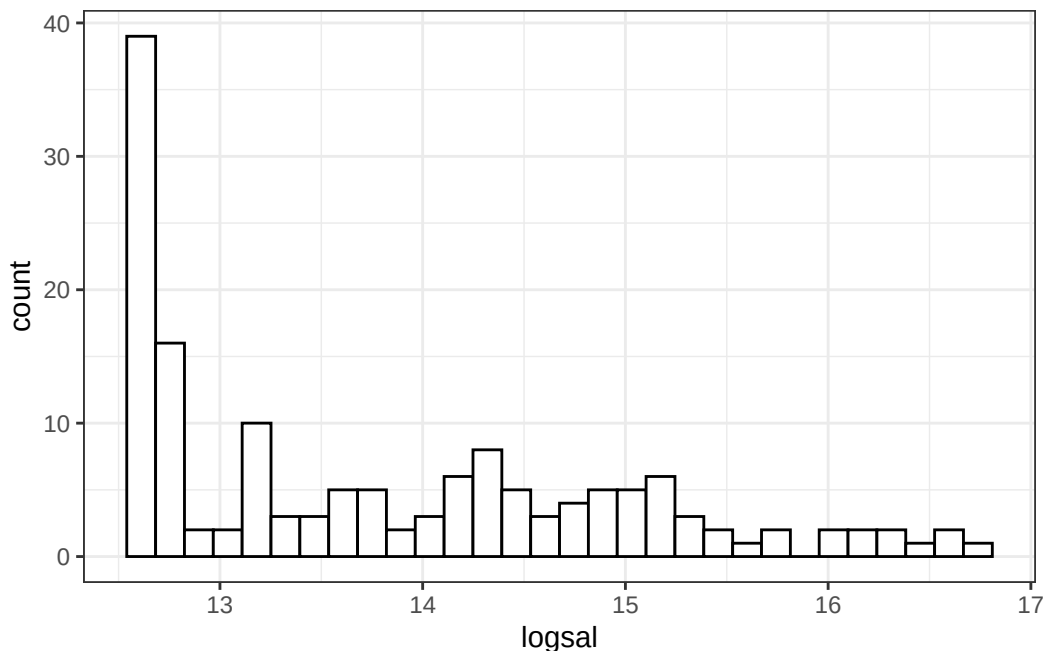
```
ggplot(data= baseball_srs, aes(x=salary))+
  geom_histogram(fill = "white",col = "black")+
  theme_bw()+
  geom_vline(xintercept = c(762500,2403125), col = "red",
             linetype = "dashed")
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



```
ggplot(data= baseball_srs, aes(x=logsal))+
  geom_histogram(fill = "white",col = "black")+
  theme_bw()
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



Describe the shape of the distribution of `salary`. Describe the shape of the distribution of `logsal`. Did taking the logarithm of salary improve the symmetry of its distribution? Explain here:

The distribution of salary is strongly right skewed, with a small number of very large salaries pulling the mean toward higher values. Most observations (75%) are below \$2.5 million, while the median salary is \$760,000.

Taking the logarithm of salary reduced the influence of these extremely large values and made the distribution more symmetric, although there is still a large concentration of observations at lower salary ranges. With this improved symmetry, the Central Limit Theorem (CLT) is likely to provide a better approximation for estimators based on sample means and totals. With 150 samples, we have a fairly large sample.

For fun, we can try the formula from the textbook (Sugden et al (2000)):

```
# without log transformation
deviations_orig <- baseball$salary - mean(baseball$salary)
28 + 25*(sum(deviations_orig^3)/(nrow(baseball)*sd(baseball$salary)^3))^2
```

```
[1] 166.7455
```

```
# with log transformation
deviations_log <- log(baseball$salary) - mean(log(baseball$salary))
28 + 25*(sum(deviations_log^3)/(nrow(baseball)*sd(log(baseball$salary))^3))^2
```

```
[1] 35.47879
```

We can see that the Sugden et al (2000) recommends a sample of 167 if we were to use the untransformed salary values. However, if we were to use the log values, we can see that the formula now says we only need a sample of 36. So we should be in really good shape to use the CLT as an approximation for the sampling distributions of the following estimators.

Notice I did this calculation using the population values. This would be good to do in the design stage of your survey if you have an idea of the skewness of your population.

Exercise 4: Inference for the population mean

Estimate the population mean of `logsal` and give a 95% CI, using your SRS. Do this manually using the formulas from lecture/the textbook — do not rely on pre-built R functions except for `mean()`, `sd()`, `qnorm(0.975)`, or `qt(0.975, df = n - 1)`, which give you the 97.5th quantile from a standard normal distribution and a t-distribution with $n-1$ degrees of freedom, respectively.

```
# Include your code here

ymean_S = mean(baseball_srs$logsal)

# here I will assume I don't know the population SD and use
# the estimate for standard error in my calculations.

s <- sd(baseball_srs$logsal)
V_hat_sqr <- s/sqrt(150)*sqrt(1-150/nrow(baseball))

ci_lwr = ymean_S - qt(0.975, df = 149)*V_hat_sqr
ci_upr = ymean_S + qt(0.975, df = 149)*V_hat_sqr

ci_lwr
```

```
[1] 13.63388
```

```
ci_upr
```

```
[1] 13.97301
```

Give the estimate and the 95% CI and interpret them here:

A 95% confidence interval for the log salary is [13.63, 13.97]. This means that we are 95% confident that the true population mean for the log of salary is contained in this interval.

Exercise 5: Inference for the population proportion

`table()` uses cross-classifying factors to build a contingency table of the counts at each combination of factor levels.

```
table(baseball$leagueid)
```

```
AL  NL  
372 425
```

Estimate the population proportion of players who are pitchers, and give a 95%, using your SRS. Do this manually as well. Type `?baseball` in your console to learn more about the variables in the dataset and identify the relevant variable.

```
# Include your code here
```

```
table(baseball_srs$pos)
```

```
1B 2B 3B  C CF LF  P RF SS  
10  7  9 17  5 11 71  9 11
```

```
p_hat_S <- 71/150  
fpc = 1- 150/nrow(baseball)  
  
# Computing this assuming we dont know the population  
# variance  
V_hat_sqr = sqrt(p_hat_S*(1-p_hat_S)/(150-1)*fpc )  
  
ci_upr = p_hat_S + qt(0.975, df=149)*V_hat_sqr  
ci_lwr = p_hat_S - qt(0.975, df=149)*V_hat_sqr  
  
ci_upr
```

```
[1] 0.5461568
```



```
ci_lwr
```

```
[1] 0.4005099
```

Give the estimate and the 95% CI and interpret them here:

A 95% confidence interval for the proportion of pitchers is $[0.4, 0.55]$ and we are 95% confident that the true population proportion of pitchers is in this interval.

Exercise 6: Conclusions

Since you have the full dataset for the population, you can find the true mean and proportion for the population.

```
# Include your code here
```

```
length(which(baseball$pos == "P"))
```

```
[1] 376
```

```
p = length(which(baseball$pos == "P"))/nrow(baseball)
```

```
p
```

```
[1] 0.4717691
```

```
mean(log(baseball$salary))
```

```
[1] 13.92706
```

Do your CIs in exercises 4 and 5 contain the true population values? What does it mean if one of your intervals misses the true value?

The true proportion of pitchers is 0.47 which is contained in the 95% CI that we constructed $[0.4, 0.55]$ based on this specific SRS sample. The population mean for log salary is 13.93, which again is contained in our realized confidence interval $[13.63, 13.97]$ from this specific SRS sample.

If one of the 95% confidence intervals didn't capture the true population mean or population proportion, this just doesn't invalidate our approach. Before we took the sample, we knew that there was a 5% probability that our confidence interval wouldn't contain the true population parameter. It appears in this case that we got lucky.